

Ensemble-Based Anomaly Detection and Health Monitoring for EV Charging Infrastructure

Technical Report

Hesam Vahib

Real-World Predictive Maintenance for EV Charger Telemetry

Abstract

This report describes an anomaly-detection and health-monitoring pipeline for EV charging infrastructure. The pipeline starts with multivariate charger telemetry, builds rolling time-series features, trains a set of supervised models and an unsupervised baseline, and then converts the final ensemble score into a health score and alert status. The main task is current anomaly detection. A separate early-warning task is treated as a future extension.

The test results are consistent with a practical maintenance setting. The selected ensemble reached about 0.93 precision, 0.86 recall, and 0.89 F1-score, with a false-alarm rate below 1%. The main drivers of abnormal behavior were module temperature imbalance, temperature rise relative to ambient conditions, voltage instability, current mismatch, and module-level power imbalance. The report also documents label construction, leakage control, threshold calibration, confidence intervals, and reproducibility notes, since these details decide whether this kind of model can be trusted.

Keywords: EV charging, predictive maintenance, anomaly detection, ensemble learning, health monitoring, charger telemetry, explainable machine learning

1. Introduction

EV charging infrastructure has become a reliability-critical part of transport electrification. Station failures and poor availability can affect user waiting time, confidence in charging services, and the operational workload of charging-network operators. Prior work has shown that charging-station reliability and availability are recurring concerns in public charging infrastructure, including user-reported service gaps across regions.

Charger abnormal behavior is not always visible as an immediate hard failure. It may first appear as thermal deviation, unstable voltage, current-delivery mismatch, communication irregularity, or uneven module-level behavior. Data-driven thermal modelling and anomaly-detection studies on EV charging stations show that charger telemetry can be used for condition monitoring and predictive-maintenance support, especially around power-converter thermal behavior.

This report treats the charger as a monitored cyber-physical asset. The aim is not to replace technician judgement or claim direct root-cause diagnosis. The aim is narrower: detect abnormal operating states from operational telemetry and convert them into a readable maintenance signal. The pipeline therefore uses session-safe splitting, rolling time-series features, calibrated thresholds, health scoring, and feature-based explanations.

2. Data and Label Construction

2.1 Dataset characterization

The evaluation uses a private full EV charger telemetry dataset stored locally and excluded from version control. The run reported here contains 100,000 telemetry records. Each row represents a sampled operating state. The sampling interval is 10 seconds, so short rolling windows can be interpreted directly in time units. The dataset covers 623 charging sessions and includes station, charger, EVSE, connector, electrical, thermal, communication, and module-level features. A small demonstration file is included in the public repository only to make the code runnable.

Table 1. Dataset characterization used in the current-anomaly experiment.

Item	Value
Total telemetry records	100,000
Charging sessions	623
Stations / chargers	50 stations / 50 chargers
Sampling interval	10 seconds
Approximate time span	13 days
Voltage categories	400 V and 800 V classes
Charging protocols	DIN70121 and ISO15118
Main modelling target	label_current_anomaly
Secondary target family	5-, 10-, and 15-minute future anomaly labels

2.2 Label construction

The label used in the main experiment is label_current_anomaly. We defined it as an operational anomaly indicator. It is not a confirmed hardware-failure label. That distinction matters. In practice, maintenance teams often need to flag abnormal operation before a root cause is known.

A record was marked anomalous when the telemetry showed evidence from one or more operational groups. The groups are listed in Table 2. The purpose of this construction was to separate ordinary charging variation from states that deserve engineering attention.

Table 2. Operational evidence used to construct anomaly labels.

Evidence group	Examples	Interpretation
Thermal abnormality	high module temperature gap; temperature-to-ambient deviation	Possible cooling issue, localized heating, or abnormal load sharing
Electrical instability	voltage variability; current-demand mismatch	Control or delivery behavior differs from expected operation
Module imbalance	pmc_temp_gap, pmc_current_gap, pmc_powerW_gap	Power modules are not sharing load or heat evenly
Communication irregularity	missing-data ratio; communication dropout indicators	Telemetry stream or backend connection is unstable
State inconsistency	charging state conflicts with measured behavior	The reported state and measured operation do not match
Severe explicit condition	critical status or failure-like indicators, where available	Immediate abnormal state used for label construction, not as a normal feature

2.3 Leakage-control strategy

The model should not learn the answer from direct status flags. We therefore kept label-like fields out of the ordinary feature matrix. These fields may be useful for label construction or reporting, but using them as model inputs would make the evaluation too easy.

Table 3. Leakage-control strategy used before model training.

Field type	Examples	Treatment
Target columns	label_current_anomaly, label_future_anomaly_5min, label_future_anomaly_10min	Excluded from features
Direct status or alarm fields	status, chargingState, hwFailure, alarms_count, errors_count	Excluded from ordinary ML features; only for label construction or reporting
Scenario or construction metadata	scenario, label_source, anomaly_start_relative_s	Excluded
Identifiers	sessionId, chargerId, stationId, connectorId	Used for grouping and splitting; not used as default predictors
Physical telemetry	current, voltage, powerW, temperature, module gaps	Retained as predictive features
Rolling operational features	voltage_roll_std_6, pmc_temp_gap_roll_mean_6, current_error_ratio	Retained as predictive features

3. Feature Engineering and Split Design

3.1 Rolling features

The suffix _6 means a six-sample rolling window. With a 10-second sampling interval, this corresponds to 60 seconds of recent charger behavior. We used this window as a first compromise: short enough to react to abnormal changes, but long enough to reduce random point-to-point noise.

The rolling features include means, standard deviations, slopes, gap measures, and mismatch ratios. This lets the model see both the current operating state and the local trend. A single voltage value may be harmless. A rising voltage standard deviation over a minute is more informative.

Table 4. Examples of engineered features.

Feature family	Examples	Reason for inclusion
Electrical variation	voltage_roll_std_6, current_slope_6, powerW_slope_6	Tracks instability and changing delivery behavior
Demand mismatch	current_error, current_error_ratio	Compares requested and measured current
Thermal deviation	temperature_to_ambient_delta, boardTemp_roll_std_6	Captures abnormal heat relative to environment
Module imbalance	pmc_temp_gap_roll_mean_6, pmc_current_gap_roll_mean_6	Detects uneven module behavior over time
Data quality	missing_ratio_roll_mean_6, missing_ratio_slope_6	Captures communication or sampling issues

3.2 Session-safe splitting

The split was done by charging session. This prevents adjacent telemetry rows from the same session appearing in both training and testing. It is a small detail, but it changes the credibility of the result.

Table 5. Train, validation, and test split.

Split	Rows	Sessions	Positive rate
Training	70,397	436	8.83%
Validation	15,534	93	9.42%
Test	14,069	94	10.31%

4. Modelling Approach

4.1 Model set

The model set was deliberately simple. Logistic Regression gives a linear reference point. Random Forest and Extra Trees capture nonlinear interactions without much preprocessing. Isolation Forest gives an unsupervised baseline. The final score combines the supervised ensemble with a small unsupervised component.

Table 6. Model configuration used in the current pipeline.

Model	Main configuration	Role in the pipeline
Logistic Regression	standardized numeric features; one-hot categorical features; solver = liblinear; class-weight handling	Linear supervised baseline
Random Forest	tree ensemble; class-weight handling; validation threshold selected from probability scores	Main nonlinear supervised model
Extra Trees	randomized tree ensemble; validation threshold selected from probability scores	High-precision nonlinear comparator
Isolation Forest	trained on normal training rows; score normalized before combination	Unsupervised anomaly baseline
Supervised weighted ensemble	weighted combination of supervised probabilities	Primary supervised ensemble score
Final ensemble	0.85 supervised ensemble + 0.15 Isolation Forest score	Selected anomaly score for health monitoring

4.2 Threshold calibration and health score

We did not use a fixed threshold such as 0.5 by default. The threshold was selected on the validation set. Two modes were kept because the operating choice depends on maintenance policy. A maintenance team may prefer higher recall during commissioning and fewer false alarms during normal operation.

The health score is calibrated from the ensemble anomaly score using the selected warning and critical thresholds. Scores below the warning threshold are mapped to the Normal range, scores between the warning and critical thresholds are mapped to the Warning range, and scores above the critical threshold are mapped to the Critical range. This keeps the health status consistent with the selected alert threshold.

Table 7. Balanced and conservative alert modes on the test set.

Mode	Threshold	Precision	Recall	F1	False alarm rate	Alert rate
Balanced	0.4915	0.9273	0.8621	0.8935	0.0078	0.0958
Conservative	0.4980	0.9285	0.8593	0.8926	0.0076	0.0954

5. Results

5.1 Model comparison

Table 8 reports the current-anomaly results on the held-out test sessions. The supervised models are close to each other. The supervised weighted ensemble has the highest F1-score, while Random Forest gives the highest precision among the main models. Isolation Forest is clearly weaker, which is expected because it does not use the operational labels.

Table 8. Model performance on the test set.

Model	Accuracy	Precision	Recall	F1	False alarm rate	ROC-AUC	PR-AUC
Supervised weighted ensemble	0.9797	0.9242	0.8745	0.8987	0.0082	0.9596	0.9289
Random Forest	0.9794	0.9354	0.8593	0.8958	0.0068	0.9590	0.9251
Final ensemble	0.9788	0.9273	0.8621	0.8935	0.0078	0.9501	0.9220
Logistic Regression	0.9780	0.9204	0.8607	0.8895	0.0086	0.9625	0.9036
Extra Trees	0.9751	0.9575	0.7931	0.8676	0.0040	0.9360	0.8894
Isolation Forest	0.8281	0.3075	0.5338	0.3902	0.1381	0.7829	0.3079

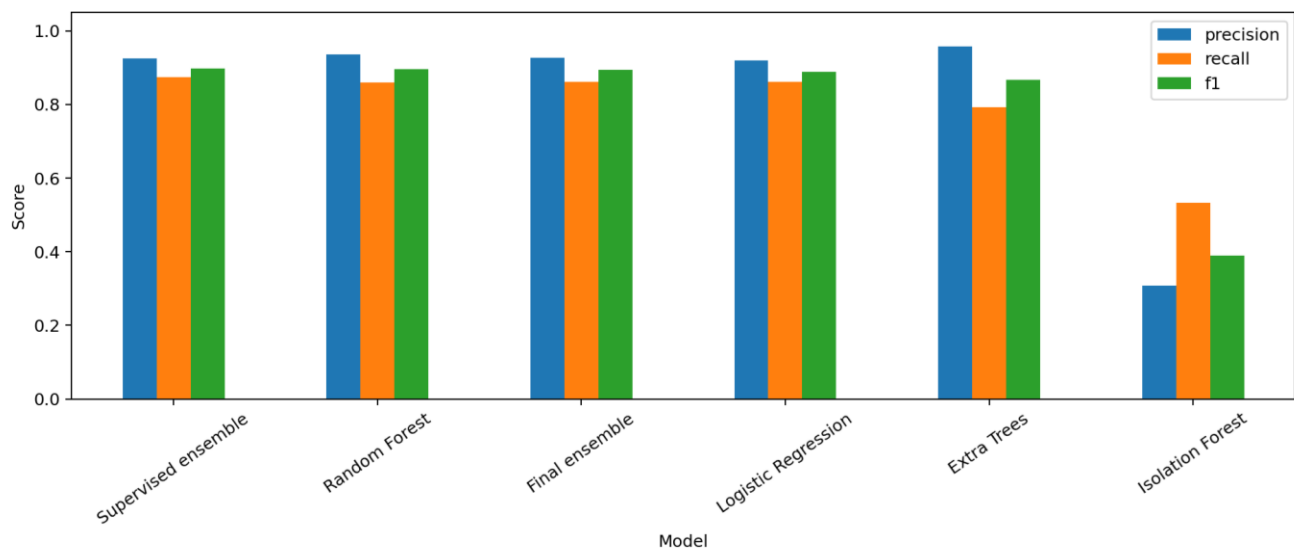


Figure 1. Model comparison using precision, recall, and F1-score.

5.2 Confusion matrix

The selected ensemble made 98 false alarms among 12,619 normal test rows. It missed 200 anomaly rows and detected 1,250. This is the practical result: the model catches most abnormal states while keeping the alert burden low.

Table 9. Confusion matrix for the selected ensemble.

True class	Predicted normal	Predicted anomaly
Normal	12,521	98
Anomaly	200	1,250

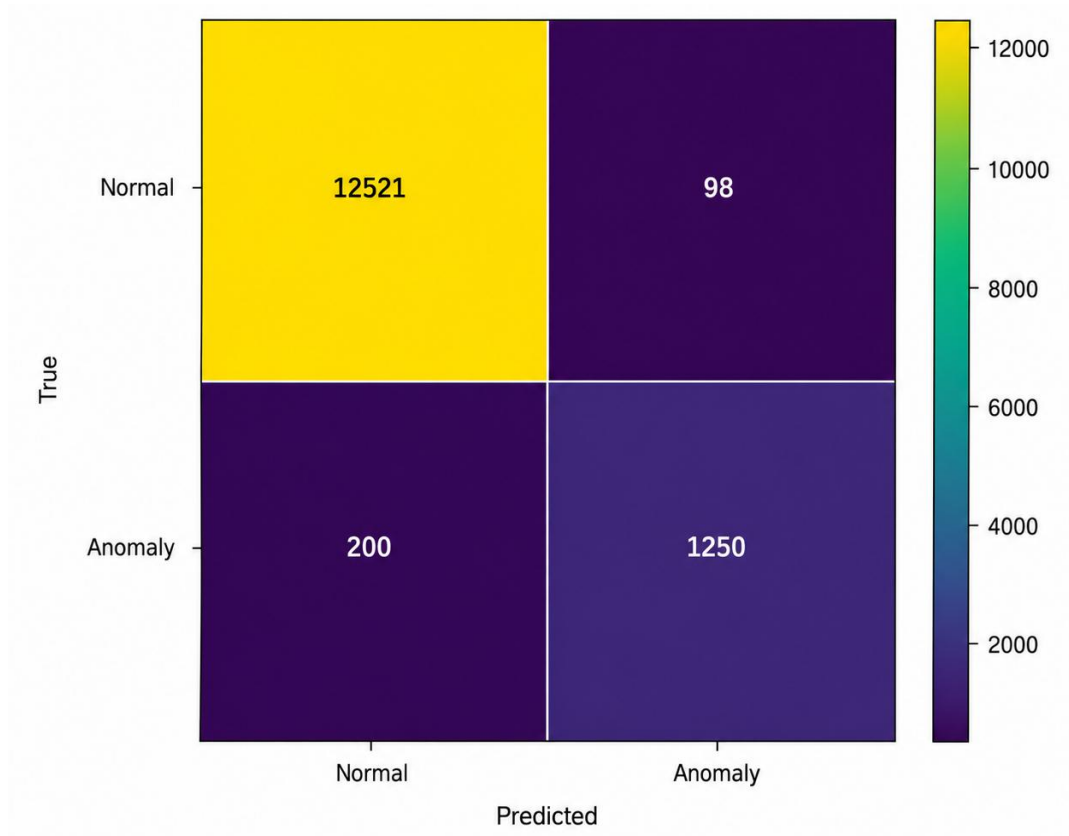


Figure 2. Confusion matrix for the selected ensemble.

5.3 Anomaly-score distribution

The score distribution is useful because it shows whether the threshold is working in a stable region. Normal rows cluster on the low-score side. Anomaly rows are mostly high-score. There is still overlap, but the separation is enough for calibrated warning and critical thresholds.

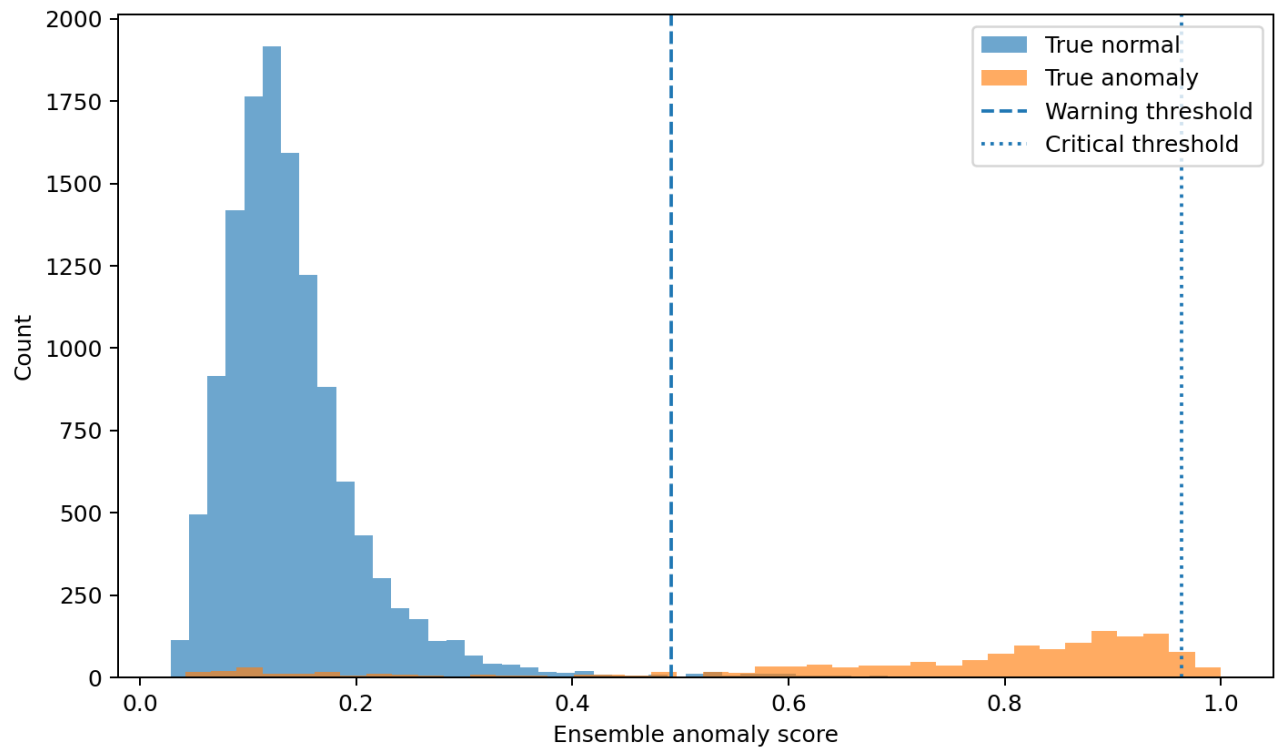


Figure 3. Ensemble anomaly-score distribution.

5.4 Session-level score behavior

The representative session shows the kind of output that a maintenance user can read quickly. The anomaly score stays low at the beginning, then rises sharply. The health score tells the same story in the opposite direction: high at first, then Warning, then Critical. This is easier to use than a stream of binary labels.

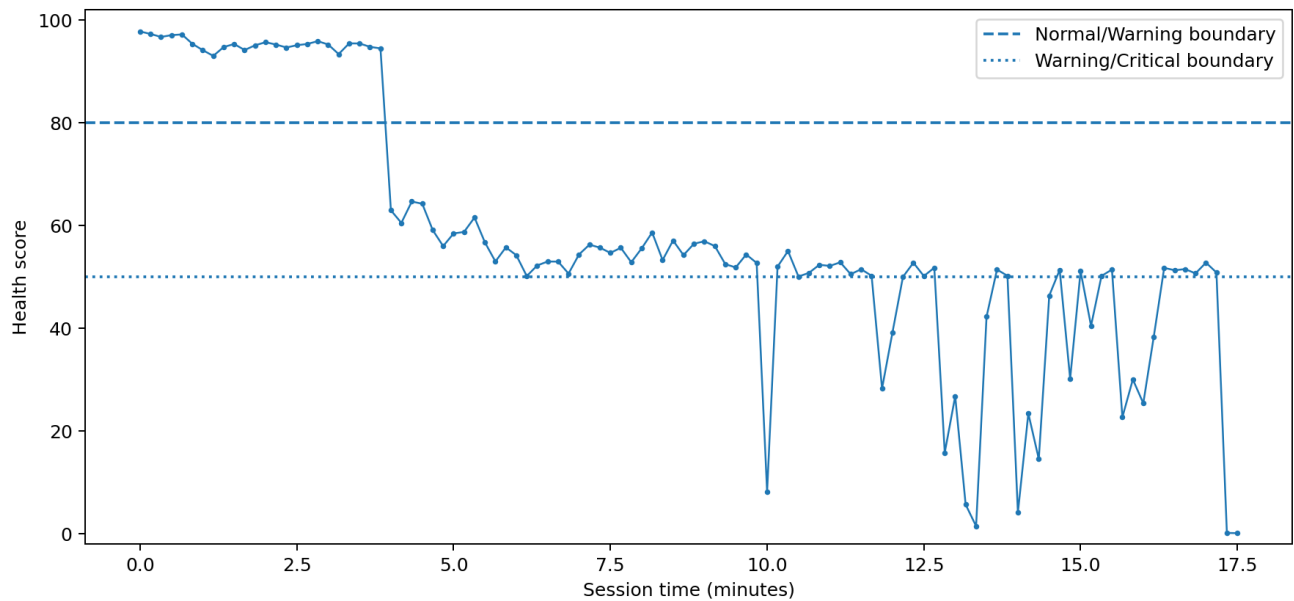


Figure 4. Health-score timeline for a representative high-risk session.

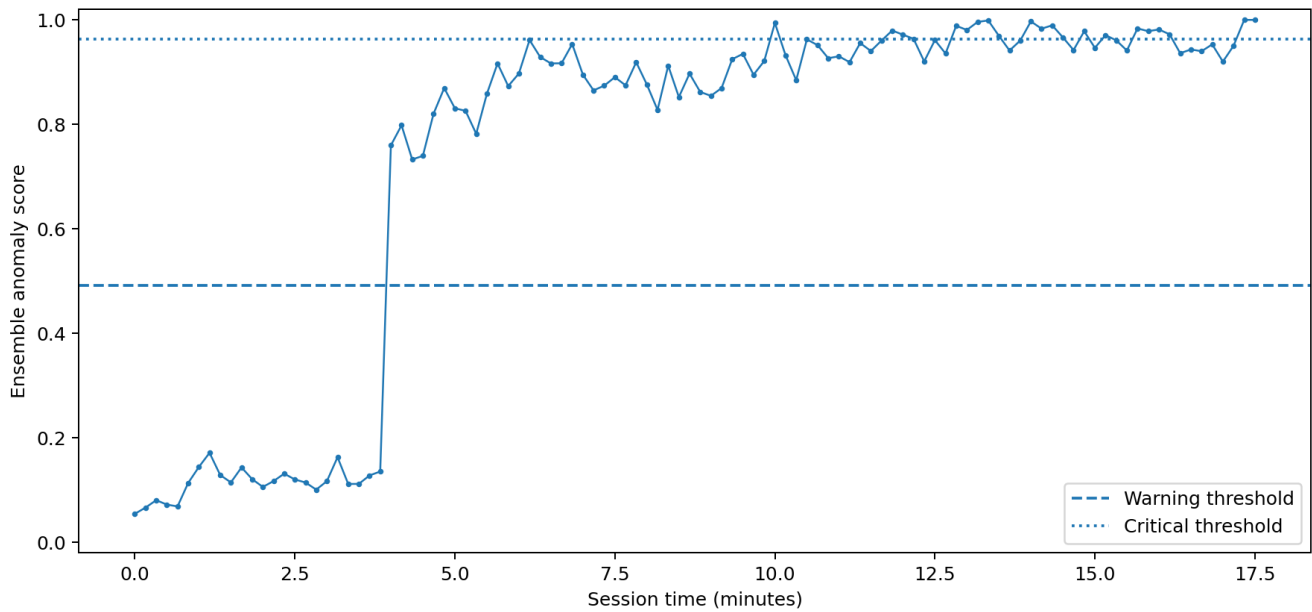


Figure 5. Anomaly-score timeline for the same representative high-risk session.

5.5 Session-level uncertainty

The point estimates are not the whole story. To estimate variation, we bootstrapped the test set by session. The recall interval is wider than the precision interval, which is reasonable because anomaly sessions are rarer and more variable.

Table 10. Session-level bootstrap confidence intervals for the selected ensemble.

Metric	Point estimate	95% confidence interval
Precision	0.927	[0.859, 0.973]
Recall	0.862	[0.733, 0.963]
F1-score	0.894	[0.795, 0.967]

5.6 Computational notes

The current implementation is an offline analysis pipeline, not yet integrated into a live monitoring service. Still, the model family is lightweight. Tree-based and linear models score rows quickly once features are available. We timed training, but did not yet benchmark per-record inference latency. That benchmark should be added before a real-time deployment claim.

Table 11. Computation summary from the current run.

Item	Value / note
Logistic Regression fit time	7.43 seconds
Random Forest fit time	3.43 seconds
Extra Trees fit time	1.53 seconds
Test rows scored	14,069
Per-observation inference latency	Not separately benchmarked in this version

6. Explainability

The explanation layer uses two views. The first is Random Forest feature importance, which gives a global picture of what the model uses. The second is a local abnormal-feature ranking based on deviation from the normal training distribution. It is simpler than SHAP, but it is transparent and easy to inspect.

Table 12. Top features and maintenance interpretation.

Rank	Signal (raw feature)	Interpretation
1	60-s average module temperature imbalance (pmc_temp_gap_roll_mean_6)	Sustained temperature imbalance between power modules
2	Module temperature imbalance (pmc_temp_gap)	Instantaneous module temperature gap
3	Temperature rise above ambient (temperature_to_ambient_delta)	Heat rise relative to ambient conditions
4	60-s voltage instability (voltage_roll_std_6)	Short-term voltage instability
5	60-s average module current imbalance (pmc_current_gap_roll_mean_6)	Sustained current imbalance between modules
6	Variation in module current imbalance (pmc_current_gap_roll_std_6)	Variation in module current imbalance
7	60-s average module power imbalance (pmc_powerW_gap_roll_mean_6)	Sustained module power imbalance
8	Current tracking error (current_error)	Difference between demanded and measured current

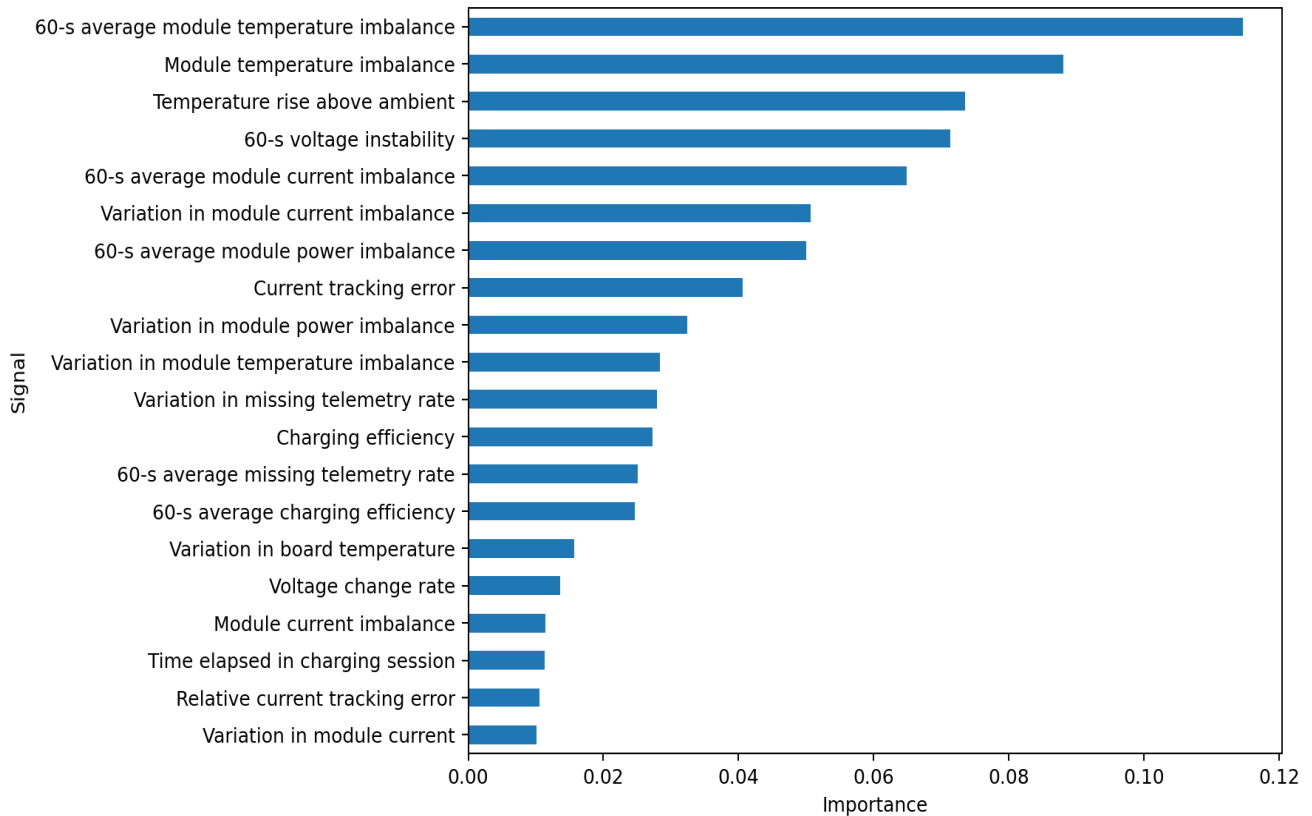


Figure 6. Most important signals used by the Random Forest model.

The high-risk rows tell a similar story. The most frequent top explanation is 60-second average module temperature imbalance. This does not mean it is the only important signal; the feature-importance view shows that thermal imbalance, voltage instability, current mismatch, and module-level power imbalance all contributed to the model decision.

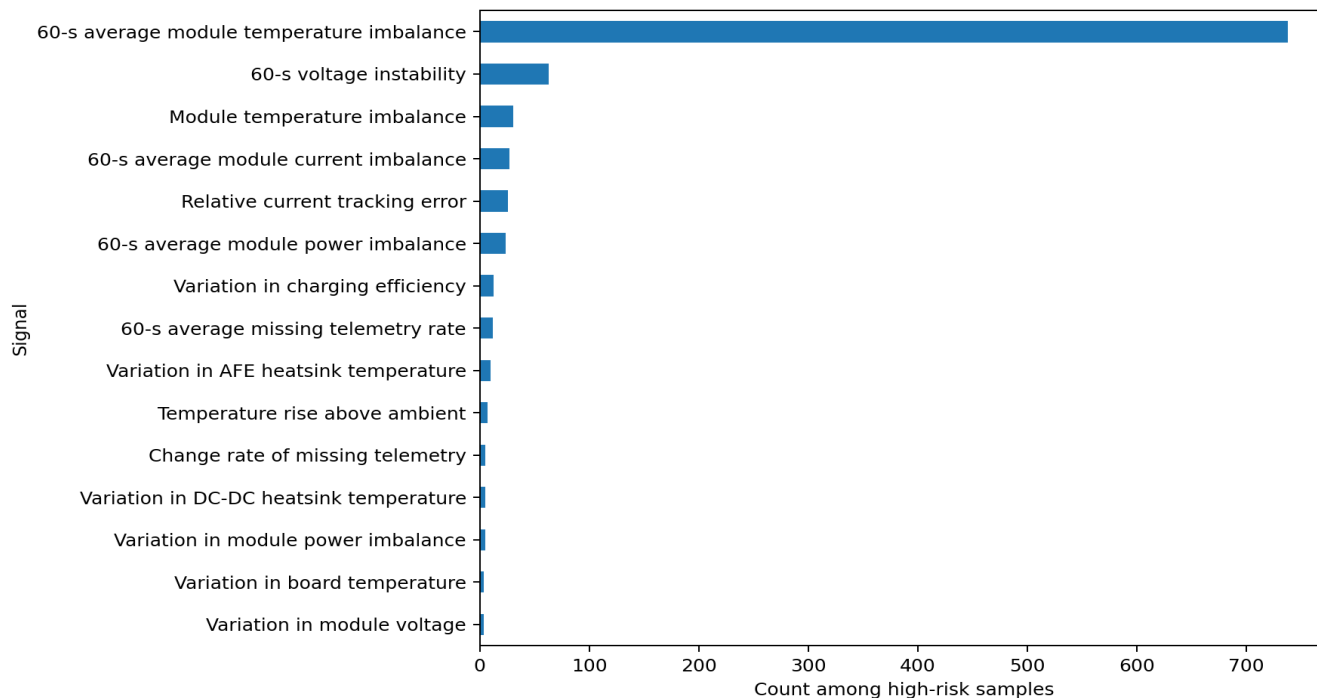


Figure 7. Most frequent abnormal signals among high-risk rows.

7. Maintenance-Oriented Output

A maintenance system should not stop at "anomaly = 1". It should tell the operator what changed and how severe it looks. Table 13 gives an example output from the pipeline.

Table 13. Example maintenance-oriented anomaly report.

Field	Output
Charger state	Critical
Health score	4%
Anomaly score	0.96
Top abnormal signal 1	60-s average module temperature imbalance
Top abnormal signal 2	60-s voltage instability
Top abnormal signal 3	60-s average module current imbalance
Interpretation	Sustained module temperature imbalance and voltage instability were detected.
Recommended action	Check module temperature, cooling, and load sharing before the next high-power charging session.

8. Discussion

The main result is straightforward. The pipeline detects current abnormal charger states with high precision and low false-alarm rate on unseen sessions. The result is not only a model score. It is a usable health-monitoring workflow: score, status, explanation, and a maintenance-facing output.

The model comparison also gives a useful lesson. Isolation Forest alone does not perform well here. The supervised models do better because the operational labels provide information about the types of abnormal states that matter. That said, the unsupervised baseline is still useful. In a station with little labelled data, it would be a starting point rather than a final model.

The feature analysis matches engineering expectations. Thermal module imbalance, voltage variability, current mismatch, and module power gaps are not arbitrary predictors. They are signals a charger engineer would already inspect. The model simply combines them at scale and tracks their short-term evolution.

There is a limit to what the current experiment proves. The labels are operational labels, not confirmed hardware-failure records. The pipeline therefore detects abnormal states; it does not prove root-cause diagnosis. That is an acceptable boundary for this stage of the project.

9. Limitations

The first limitation is label quality. Operational labels are useful, but they are not the same as maintenance-confirmed failure labels. Future validation should connect alerts to work orders, technician findings, replacement events, or charger downtime.

The second limitation is dataset coverage. Although the evaluation uses 100,000 telemetry records, the data cover 623 charging sessions over an approximately 13-day period. This is enough for a controlled current-anomaly evaluation, but it does not fully capture long-term seasonal effects, rare degradation patterns, charger aging, or site-specific operating differences. A larger evaluation across longer time periods and more diverse charging locations would be needed before making strong fleet-level deployment claims.

The third limitation is the temporal validation design. Session-safe splitting avoids a major leakage problem, but time-ordered validation would be a better test of deployment behavior. A later version should train on earlier sessions and test on later sessions.

The fourth limitation is explainability. Z-score ranking is clear, but it does not capture feature interactions. SHAP values and partial dependence plots would help show how risk changes as module temperature gaps or voltage variability increase.

Finally, the current report focuses on current anomaly detection. Early-warning prediction is the more valuable maintenance direction. The labels already support 5-, 10-, and 15-minute horizons, but those experiments should be reported separately to avoid mixing current-state detection with forecasting.

10. Reproducibility, Ethics, and Safety

The current implementation is organized as a small Python pipeline around operational EV charger telemetry. The public repository includes code, a small demonstration telemetry file, output figures, and result summaries. The full operational dataset is stored locally under `private_data/` and is excluded from version control. In a live deployment, access control and cybersecurity would matter, especially if alerts were connected to charger control systems or backend services.

Table 14. Reproducibility summary.

Item	Description
Main script	analysis/run_pipeline.py
Supporting modules	analysis/telemetry_data.py; analysis/anomaly_models.py
Repository demo input	sample_data/telemetry_sample_500.csv
Private full evaluation dataset	Stored locally under private_data/ and excluded from version control
Primary target	label_current_anomaly
GitHub repository	github.com/HesamVahib/ev-charger-predictive-maintenance
Main outputs	- results_current_anomaly/tables/ for CSV result tables; - results_current_anomaly/figures/ for plots; - results_current_anomaly/run_summary.md for the run summary.
Safety note	Model alerts support inspection and prioritization, not automated safety decisions.

11. Future Work

The next coding cycle should not add complexity for its own sake. It should test whether the model generalizes and whether the alert is useful before the fault becomes obvious.

- Run the 5-, 10-, and 15-minute early-warning experiments and report lead time together with false alarms.
- Benchmark per-record inference latency on a local CPU and, later, on an edge device.
- Add simple baselines: fixed thresholds, moving-average deviation, and rule-only alerting.
- Test time-ordered validation and leave-station-out validation.
- Add SHAP explanations and partial dependence plots for the top thermal and electrical features.
- Connect alerts to maintenance logs when those records are available.
- Build a small dashboard for live or replay-based charger health monitoring.

12. Conclusion

This real-world telemetry evaluation shows that ensemble models can turn charger data into a practical anomaly-detection and health-monitoring workflow. The selected ensemble reached about 0.93 precision, 0.86 recall, and 0.89 F1-score on held-out sessions, with a false-alarm rate below 1%. That is the central result.

The more important point is how the result is packaged. The system does not only output a label. It reports a calibrated anomaly score, a health score, an alert state, and the main abnormal signals behind the decision. That makes the output closer to what a maintenance engineer can use.

The work still needs maintenance-log validation, early-warning experiments, and better local explanations. Even so, the current version gives a clear foundation for predictive maintenance of EV charging infrastructure and for transfer to other telemetry-rich machinery systems.